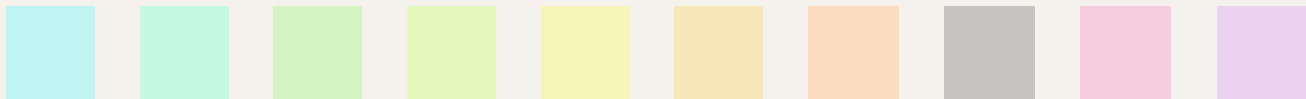


Client Side JavaScript

מבוא לבניית מערכות ממוחשבות מבוססות אינטרנט הרצאה 6 – JavaScript





Functions

JavaScript Functions

- A JavaScript function is a block of code designed to perform a particular task.
- A JavaScript function is executed when "something" invokes it (calls it).

JavaScript Function Syntax

- A JavaScript function is defined with the function keyword, followed by a name, followed by parentheses ().

```
function name(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

- Function **parameters** are listed inside the parentheses () in the function definition.
- Function **arguments** are the **values** received by the function when it is invoked.
- Inside the function, the arguments (the parameters) behave as local variables.

Function Invocation

- The code inside the function will execute when "something" invokes (calls) the function:
 1. When an event occurs (when a user clicks a button)
 2. When it is invoked (called) from JavaScript code
 3. Automatically (self invoked)

Function Return

- When JavaScript reaches a return statement, the function will stop executing.
- If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.
- Functions often compute a return value.
- The return value is "returned" back to the "caller"

```
var x = myFunction(4, 3);  
// Function is called, return value will end up in x  
  
function myFunction(a, b) {  
    return a * b;  
// Function returns the product of a and b  
}
```

Functions- exercise

- Go to : <https://stanford.edu/~cpiech/karel/ide.html>
 - Or type karel robot on google (first site)
- Change the world to 5X5
- Under references you can find all available functions of the robot
- Write a function that puts beepers on the diagonal of the board





Conditions and Loops

Conditional Statements

- In JavaScript we have the following conditional statements:
 1. Use **if** to specify a block of code to be executed, if a specified condition is true
 2. Use **else** to specify a block of code to be executed, if the same condition is false
 3. Use **else if** to specify a new condition to test, if the first condition is false
 4. Use **switch** to specify many alternative blocks of code to be executed

The if Statement

- Use the if statement to specify a block of JavaScript code to be executed if a condition is true.

- Syntax:

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

- Example:

```
if (age < 18) {  
    allow_alcohol = False;  
}
```

The else Statement

- Use the else statement to specify a block of code to be executed if the condition is false.

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

The else if Statement

- Use the else if statement to specify a new condition if the first condition is false.
- Syntax:

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```

The JavaScript Switch Statement

- The switch statement is used to perform different actions based on different conditions.
- Use the switch statement to select one of many code blocks to be executed.

```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default:  
    // code block  
}
```

The JavaScript Switch Statement

```
switch (new Date().getDay()) {  
  case 0:  
    day = "Sunday";  
    break;  
  case 1:  
    day = "Monday";  
    break;  
  case 2:  
    day = "Tuesday";  
    break;  
  case 3:  
    day = "Wednesday";  
    break;  
  case 4:  
    day = "Thursday";  
    break;  
  case 5:  
    day = "Friday";  
    break;  
  case 6:  
    day = "Saturday";  
}
```



Home exercise- conditional statements and functions

- Write a code that checks if a year is a leap year or not.
- Complete the code:
 - ```
function isLeap(year){
 //write your code here
}
```

## Rules:

A year is a leap year if it:

Divides by 4- leap,

unless- divided also by 100- then not leap,

Unless- divides also by 400- leap.

Example: 2400

$2400/4=600$ - Leap

$2400/100=24$ - Not leap

$2400/400=6$ -Leap

2400 is a leap year!

# JavaScript Loops

- Loops can execute a block of code a number of times.
- Loops are handy, if you want to run the same code over and over again, each time with a different value.
  - Often this is the case when working with arrays

# Different Kinds of Loops

- JavaScript supports different kinds of loops:
  1. while - loops through a block of code while a specified condition is true
  2. do/while - also loops through a block of code while a specified condition is true
  3. for - loops through a block of code a number of times
  4. for/in - loops through the properties of an object
  5. for/of - loops through the values of an iterable object (e.g. array, string...)

# The While Loop

- The while loop loops through a block of code as long as a specified condition is true.
- Syntax:

```
while (condition) {
 // code block to be executed
}
```

- In the following example, the code in the loop will run, over and over again, as long as a variable (i) is less than 10:

```
i=0;
while (i < 10) {
 text += "The number is " + i;
 i++;
}
```

# The Do/While Loop

- The do/while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.
- Syntax:

```
do {
 // code block to be executed
}
while (condition);
```

```
do {
 text += "The number is " + i;
 i++;
}
while (i < 10);
```

# The For Loop

- Syntax:

```
for (statement 1; statement 2; statement 3) {
 // code block to be executed
}
```

- **Statement 1** is executed (one time) before the execution of the code block.
- **Statement 2** defines the condition for executing the code block.
- **Statement 3** is executed (every time) after the code block has been executed.

# For Loop Example

```
for (var i = 0; i < 5; i++) {
 text += "The number is " + i + "
";
}
```

- From the example above, you can read:
- Statement 1 sets a variable before the loop starts (var i = 0).
- Statement 2 defines the condition for the loop to run (i must be less than 5).
- Statement 3 increases a value (i++) each time the code block in the loop has been executed.

# For Loop and While loop relations

- The for loop is basically the while loop, but with simplified syntax!
- In while loops statement 1 and 3 needs to be stated outside the loop statement, while in For loops they are inserted into the loop!

```
for (i = 0; i < 5; i++) {
 text += "The number is " + i + "
";
}
```

```
i=0;
while (i < 5) {
 text += "The number is " + i;
 i++;
}
```

# The For/Of Loop

- The JavaScript for/of statement loops through the values of an iterable objects.
- for/of lets you loop over data structures that are iterable such as Arrays or strings.
- Syntax:

```
for (variable of iterable) {
 // code block to be executed
}
```

# The For/Of Loop Through Arrays

- Example:

```
var cars = ["BMW", "Volvo", "Mini"];

for (var x of cars) {
 console.log(x);
}
```

BMW

Volvo

Mini

# The For/Of Loop Through Strings

- Example:

```
var txt = "JavaScript";

for (var x of txt) {
 console.log(x);
}
```

J

a

v

a

S

c

r

i

p

t

# Home exercise- Loops and functions

- Write a function :
  - Function fibonacciGenerator(n){
  - }
- The function receives an input indicating the length of the Fibonacci list that needs to be generated and returns the Fibonacci list.
- Reminder: Fibonacci starts with 0,1 and then every item will be the sum of the 2 previous items-
  - fibonacciGenerator(4):
    - 0,1,1,2
  - fibonacciGenerator(10):
    - 0,1,1,2,3,5,8,13,21,34





**Events**

# Events

- HTML events are "things" that happen to HTML elements.
- When JavaScript is used in HTML pages, JavaScript can "react" on these events.
- An HTML event can be something the browser does, or something a user does.
- Examples of HTML events:
  1. An HTML web page has finished loading
  2. An HTML input field was changed
  3. An HTML button was clicked
  4. User has moved the mouse cursor

# HTML Events

- HTML allows event handler attributes, with JavaScript code, to be added to HTML elements.

```
<element event='some JavaScript'>
```

# Onclick Attribute – Example

- JavaScript code changes the content of the element with id="demo", with Inline JS:

```
<button onclick="document.getElementById('demo').innerHTML = Date()">The time is?</button>
```

- Or with code that changes the content of its own element (using this.innerHTML):

```
<button onclick="this.innerHTML = Date()">The time is?</button>
```

- Or calling a function from a <script>:

```
<button onclick="displayDate()">The time is?</button>
```



# Common HTML Events

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page



Objects

# JavaScript Objects

- You have already learned that JavaScript variables are containers for data values.

```
var car = "Fiat";
```

- Objects are variables too. But objects can contain many values.
- This code assigns **many values**

```
var car = {type:"Fiat", model:"500", color:"white"};
```

- The values are written as **name:value** pairs (name and value separated by a colon).

# Creating a JavaScript Object

- With JavaScript, you can define and create your own objects.
- There are different ways to create new objects:
  1. Define and create a single object, **using an object literal**.
  2. Define and create a single object, **with the keyword new**.
  3. Define an **object constructor**, and then create objects of the constructed type.

# Object Definition with an Object Literal

- You define (and create) a JavaScript object with an object literal
- Use the **var** keyword to define the object and use curly brackets to define its properties and values:

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

# Using the JavaScript Keyword new

- The following example also creates a new JavaScript object with four properties:

```
var person = new Object();
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";
```

# Using Constructor (=a function)

- The following example creates a new JavaScript object with four properties:
- Will always start with capital letter!

```
function Person (firstName, lastName, age, eyeColor){
 this. firstName = firstName ;
 this. lastName = lastName ;
 this. age = age ;
 this. eyeColor = eyeColor ;
}
```

```
Var person1=new Person("Timmy", "Dean", 16, "blue");
```

# Accessing Object Properties

- You can access object properties in two ways:

```
objectName.propertyName
person1.firstName
```

- Or:

```
objectName["propertyName"]
person1["firstName"]
```

# Object Methods

- Objects can also have methods.
- Methods are actions that can be performed on objects.
- Methods are stored in properties as function definitions.
- A method is a function stored as a property.

```
var person = {
 firstName: "John",
 lastName : "Doe",
 id : 5566,
 fullName : function() {
 return this.firstName + " " + this.lastName;
 }
};
```



# The 'this' Keyword

- In a function definition, **this** refers to the "owner" of the function.
- In other words, `this.firstName` means the `firstName` property of this object.

# Accessing Object Methods

- You access an object method with the following syntax:

*objectName.methodName()*

- Example:

```
name = person.fullName();
```

# The For/In Loop

- The JavaScript for/in statement loops through the properties of an object
- Example:

```
var person = {fname:"John", lname:"Doe", age:25};

var text = "";
for (var x in person) {
 text += person[x];
}
```

John Doe 25



# The HTML DOM- Document Object Model

# DOM

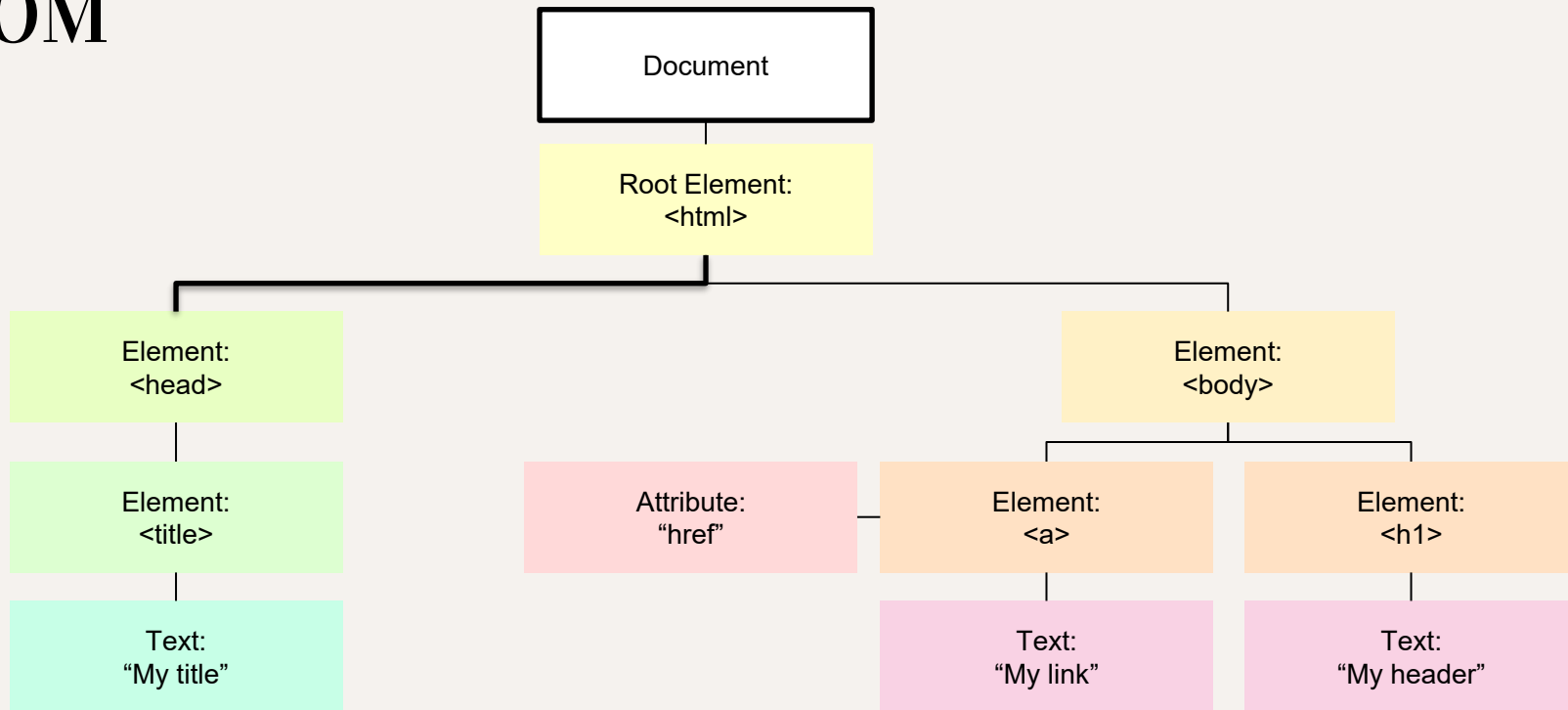
- When a web page is loaded, the browser creates a DOM **Document Object Model** of the page.
- The HTML DOM model is constructed as a tree of Objects.
- <https://chromewebstore.google.com/detail/html-tree-generator/dlbbmhhaadfnbbdnjalilhdakfmiffeg?pli=1>

# What is the HTML DOM?

- The HTML DOM is a standard object model and programming interface for HTML. It defines:
  1. The HTML elements as objects
  2. The properties of all HTML elements
  3. The methods to access all HTML elements
  4. The events for all HTML elements

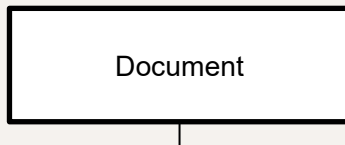
In other words: The HTML DOM is a standard for how to get, change, add, or delete HTML elements.

# DOM



# DOM

- The HTML DOM **document object** is the owner of all other objects in your web page.
- The document object **represents your web page.**
- If you want to access any element in an HTML page, you always start with accessing the document object.



- **document**.querySelector("selectorName").....

# DOM Methods and Properties

- HTML DOM methods are actions you can perform (on HTML Elements).
- HTML DOM properties are values (of HTML Elements) that you can set or change.

```
document.getElementById('demo').innerHTML
```

Object . Method() . Property

# The DOM Programming Interface

- The HTML DOM can be accessed with JavaScript (and with other programming languages).
- In the DOM, all HTML elements are defined as objects.
- The programming interface is the properties and methods of each object.

# Finding HTML Elements

- Finding HTML elements by id
- Finding HTML elements by tag name
- Finding HTML elements by class name
- Finding HTML elements by CSS selectors
- Finding HTML elements by HTML object collections

# Finding HTML Elements

Method	Description
<code>document.getElementById(id)</code>	Find an element by its id
<code>document.getElementsByTagName(name)</code>	Find elements by tag name
<code>document.getElementsByClassName(name)</code>	Find elements by class name

# Changing HTML Elements

- With DOM you can change html elements:
  - Content
  - Attribute
  - properties

# Changing HTML Elements

Property	Description
<code>element.innerHTML = new html content</code>	Change the inner HTML of an element
<code>element.attribute = new value</code>	Change the attribute value of an HTML element
<code>element.style.property = new style</code>	Change the style of an HTML element
Method	Description
<code>element.setAttribute(attribute, value)</code>	Change the attribute value of an HTML element

# Adding Events Handlers

- With DOM you can add events to any element you want.
- You can add multiple events to a single element.
- You can even add multiple functions to a single event.



# JavaScript HTML DOM EventListener

- The addEventListener() method

```
document.getElementById("myBtn").addEventListener("click",
displayDate);
```

# The addEventListener() method

- The addEventListener() method attaches an event handler to an element without overwriting existing event handlers.
- You can:
  1. add many event handlers to one element.
  2. add many event handlers of the same type to one element, i.e two "click" events.
  3. add event listeners to any DOM object not only HTML elements. i.e the window object.

# The `addEventListener()` method

- The `addEventListener()` method makes it easier to control how the event reacts to bubbling.
- When using the `addEventListener()` method, the JavaScript is separated from the HTML markup, for better readability and allows you to add event listeners even when you do not control the HTML markup.
- You can easily remove an event listener by using the `removeEventListener()` method.

# The `addEventListener()` method - Syntax

```
element.addEventListener(event, function, useCapture);
```

- The first parameter is the type of the event (like "click" or "mousedown" or any other HTML DOM Event.)
- The second parameter is the function we want to call when the event occurs.
- The third parameter is a boolean value specifying whether to use event bubbling or event capturing. This parameter is optional.

# Finding HTML Objects

- The following HTML objects (and object collections) are also accessible:
  - `document.anchors`
  - `document.body`
  - `document.documentElement`
  - `document.embeds`
  - `document.forms`
  - `document.head`
  - `document.images`
  - `document.links`
  - `document.scripts`
  - `document.title`

# Finding HTML Objects

- Examples:

Property	Description
document.forms	Returns all <form> elements
document.head	Returns the <head> element
document.images	Returns all <img> elements
document.scripts	Returns all <script> elements
document.title	Returns the <title> element
document.URL	Returns the complete URL of the document